Data Management - Project Report

Vlad Turno (1835365)

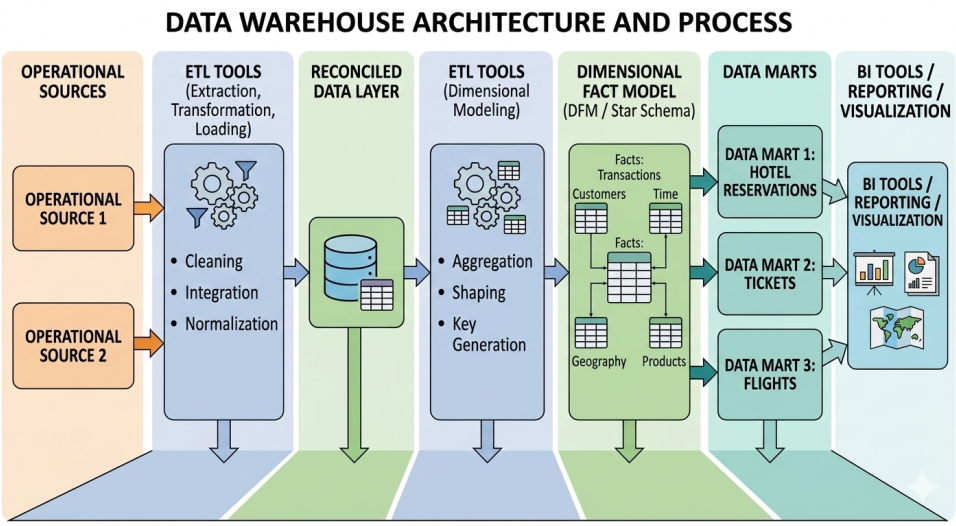
March 8, 2026

Data Warehousing

The following document aims to illustrate the process of design and deployment of a data warehouse, implemented adopting a three-layer architecture as shown in the figure below. The data processing pipeline is divided into four main steps:

- Extracting the raw operational data from the source layer
- Populating a reconciled data layer after cleansing and filtering the operational data
- Converting the reconciled data to a dimensional fact model to populate the actual warehouse
- Feeding the warehouse data to a business intelligence in order to carry on some specific analysis involving different aspects about large amounts of data

The three-layer architecture is made by a source layer featuring operational data from different organizations, a reconciled data layer unifying the view for the heterogeneous operational data and a warehouse made by multidimensional modeling of the reconciled data. The primary warehouse features an ulterior subdivision into thematic data marts, in order to reduce the scope of the business intelligence tools for better ease of usage.



Operational Data

The source layer is made by operational data coming from two different corporations called *Meridiano* and *Conti Travel*. They both operate in the tour operator field and they feature a similar infrastructure and logical organization of the data since, back in the day, the management application was developed by the same company (I'm working with, ndr).

Despite a few differences depending on the corporation, the data structure features the typical elements of a travel agency: they have to deal with flight booking, train tickets, hotel reservations, management of travel packages and promotions, an anagraphic for tour operators and clients, payment/billing stuff and all things related to those field.

To have a measure of the database size, think about the fact that companies have been operating for a few decades now and some tables (like the ticket ones) can easily reach and overtake the half-a-million rows. It is not that much, but surely is enough to make some meaningful analysis with statistical relevance.

Talking about the backend schema it is sometimes a bit messy and poorly designed, surely also due to factors among which I imagine the integration of new requests over time and unclear system requirements between different development iterations.
In the following page is reported a clear example of some tables with many fields having cryptic meaning and dubious utility.

One last observation concerns some (understandable) limitations given to me by the corporations:

- Anonymize the personal data of every guest/passenger reported within any reservation/ticket table
- Do not bring outside of the corporation's database anything related to billing addresses and payment information

So let it be right away, also because the number of tables being not high but not negligible either, it's better this way.

AEROPORTI

AEROPORTI_ID
AEROPORTI_COD
AEROPORTI_DES
AEROPORTI_AREE
AEROPORTI_NAZIONI
AEROPORTI_PRINC
AEROPORTI_PART
AEROPORTI_PARTV
AEROPORTI_PICKUPV
AEROPORTI_PICKUPT
AEROPORTI_PICKUPN
AEROPORTI_PICKUPA
AEROPORTI_CITPRIH
AEROPORTI_CITPRIS
AEROPORTI_CITPRIV
AEROPORTI_CITPRIP
AEROPORTI_REGIONE
AEROPORTI_DESTINAZIONE

TKT

Tkt_id
Tkt_idresa
Tkt_resa
Tkt_data
Tkt_anno
Tkt_num
Tkt_Tbigl
Tkt_Serie
Tkt_NTkt
Tkt_Cong
Tkt_NTktC
Tkt_NTktA
Tkt_El
Tkt_Stock
Tkt_Cassa
Tkt_Pnr
Tkt_NomePax
Tkt_CCosto
Tkt_Commessa
Tkt_Matricola
Tkt_Causale
Tkt_Descrizione
Tkt_AutorizzazioneC
Tkt_NomeP
Tkt_IndirizzoP
Tkt_CapP
Tkt_CittaP
Tkt_ProvinciaP
Tkt_PivaP
Tkt_TelefonoP
Tkt_FaxP
Tkt_emailP
Tkt_age
Tkt_cli
Tkt_Nomecli
Tkt_tara
Tkt_taru
Tkt_tarp
Tkt_tax
Tkt_ibonus
Tkt_nbonus
Tkt_ImpLordo
Tkt_Provvigione
Tkt_commissione
Tkt_perP
Tkt_perC
Tkt_Fee
Tkt_FC
Tkt_fp
Tkt_datadal
Tkt_dataal
Tkt_dataopz
Tkt_nopz
Tkt_Vettore
Tkt_EBSP
Tkt_NIC
Tkt_Area
Tkt_Tariffa
Tkt_Cit1
Tkt_Cit2
Tkt_Cit3
Tkt_Cit4
Tkt_Cit5
Tkt_destp
Tkt_CitP
Tkt_Cit1D
Tkt_Cit2D
Tkt_Cit3D
Tkt_Cit4D
Tkt_Cit5D
Tkt_Citta
Tkt_Cfm
Tkt_NazHotel
Tkt_CodHotel
Tkt_NomeHotel
Tkt_pax
Tkt_notti
Tkt_Sis
Tkt_Tratt
Tkt_Export
Tkt_Dataexport
Tkt_codoperatore
Tkt_descoperatore
Tkt_ageoperatoreA
Tkt_codoperatoreA
Tkt_descoperatoreA
Tkt_ann

HOTEL

HOTEL_ID
HOTEL_COD
HOTEL_AREA
HOTEL_NAZ
HOTEL_KEY
HOTEL_CIT
HOTEL_CITC
HOTEL_DES
HOTEL_PACC
HOTEL_IMM
HOTEL_CITTA
HOTEL_BACKUP
HOTEL_CAT
HOTEL_IND
HOTEL_CAP
HOTEL_DIRETTORE
HOTEL_CAPORIC
HOTEL_TITUBIC
HOTEL_UBIC
HOTEL_REGIONE
HOTEL_PR
HOTEL_TEL
HOTEL_CELLULARE
HOTEL_FAX
HOTEL_TELEX
HOTEL_VERDE
HOTEL_EMAIL
HOTEL_TEL_GR
HOTEL_CELLULARE_GR
HOTEL_FAX_GR
HOTEL_TELEX_GR
HOTEL_VERDE_GR
HOTEL_EMAIL_GR
HOTEL_BOOKING
HOTEL_C01
HOTEL_C02
HOTEL_C03
HOTEL_C04
HOTEL_C05
HOTEL_C06
HOTEL_C07
HOTEL_C08
HOTEL_C09
HOTEL_C10
HOTEL_C11
HOTEL_C12
HOTEL_C13
HOTEL_C14
HOTEL_C15
HOTEL_C16
HOTEL_C17
HOTEL_C18
HOTEL_C19
HOTEL_C20
HOTEL_C21
HOTEL_C22
HOTEL_C23
HOTEL_C24
HOTEL_C25
HOTEL_C26
HOTEL_C27
HOTEL_C28
HOTEL_C29
HOTEL_C30
HOTEL_C31
HOTEL_C32
HOTEL_C33
HOTEL_C34
HOTEL_C35
HOTEL_C36
HOTEL_C37
HOTEL_C38
HOTEL_C39
HOTEL_C40
HOTEL_TITNOTE1
HOTEL_NOTE1
HOTEL_TITNOTE2
HOTEL_NOTE2
HOTEL_PIVA
HOTEL_TITNOTE
HOTEL_NOTE
HOTEL_TITNOTE3
HOTEL_NOTE3
HOTEL_VAL
HOTEL_FORN
HOTEL_FPLAN
HOTEL_SR

Reconciled Data Layer

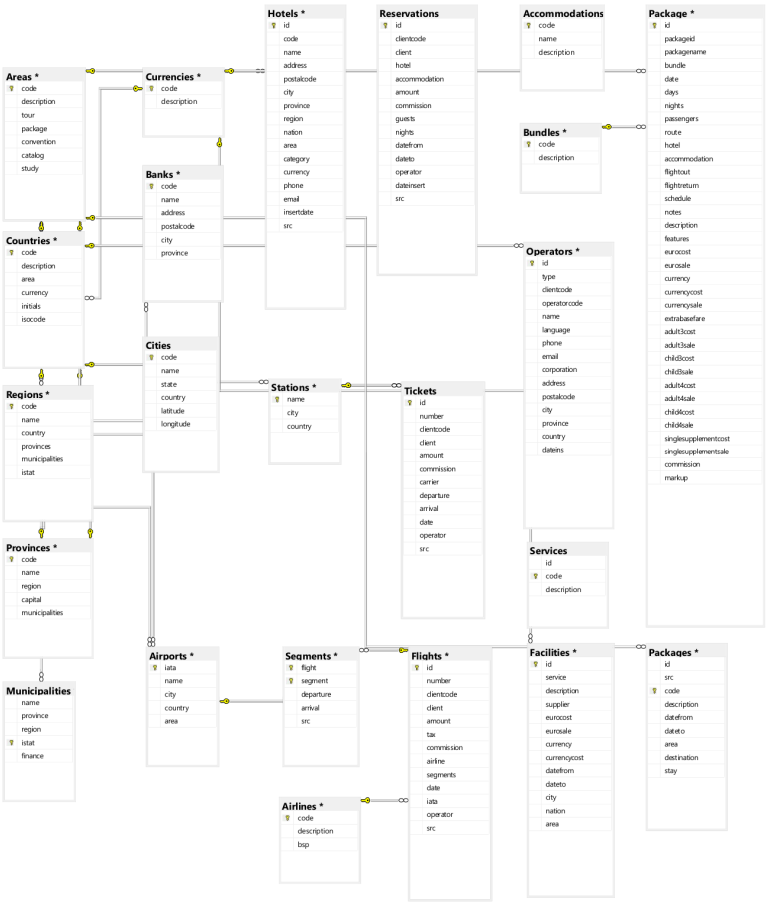
Once the operational data has been anonymized the right way and the tables of interest imported into the operational source layer, a first ETL process started to extract and clean data by reducing the number of fields in the tables, cleaning the data, removing duplicated records and so on.

Please note that due to the high number of records (hence due to the high number of possible limit cases) some inconsistencies were easy to fix (e.g. do not import a flight row where the airline field is NULL) but some others are probably still hiding somewhere (e.g. inconsistent link between client code and actual client who asked for a reservation).

An exhaustive list of the ETL queries made to create and populate the tables for the reconciled data layer can be found in the folder attached to this document, alongside with the scripting files for the whole database, the schemas and other material. In order to enrich the content of this report a couple of those queries are provided down here:

```sql
INSERT INTO reconcileddatalayer.dbo.operators (id, type, clientcode, operatorcode,
    name, language, phone, email, corporation, address, postalcode, city, province,
     country, dateins)
SELECT
    CAST(conti.id AS NVARCHAR(10)),
    CASE WHEN conti.tipo = 'Cliente' THEN 'CLIENT' ELSE 'ADMIN' END,
    CAST(conti.CodCli AS NVARCHAR(10)),
    CAST(conti.CodOperatore AS NVARCHAR(10)),
    CAST(conti.DescOperatore AS NVARCHAR(50)),
    CASE
        WHEN conti.linguacli IS NULL
        THEN (CASE WHEN conti.country IS NULL THEN '' ELSE CAST(conti.country AS
            NVARCHAR(3)) END)
        ELSE CAST(conti.Linguacli AS NVARCHAR(3)) END,
    CAST(conti.Telefono AS NVARCHAR(15)),
    CAST(conti.Email AS NVARCHAR(50)),
    CAST(conti.Denominazione AS NVARCHAR(50)),
    CAST(conti.Indirizzo AS NVARCHAR(100)),
    CAST(conti.cap AS NVARCHAR(10)),
    CAST(conti.Citta AS NVARCHAR(25)),
    CAST(conti.Provincia AS NVARCHAR(2)),
    CASE WHEN conti.country IS NULL THEN '' ELSE CAST(conti.country AS NVARCHAR(3))
         END,
    conti.Datainserimento
FROM contitravel.dbo.anagrafica AS conti
```

```sql
INSERT INTO ReconciledDataLayer.dbo.Tickets
SELECT
    CAST(t.Tkt_ID AS NVARCHAR(50)),
    CAST(t.Tkt_NTkt AS NVARCHAR(15)),
    CAST(t.Tkt_Cli AS NVARCHAR(10)),
    CAST(t.Tkt_Nomecli AS NVARCHAR(100)),
    t.Tkt_Implordo,
    t.Tkt_Provvigione,
    CAST(t.Tkt_Note AS NVARCHAR(25)),
    CAST(t.Tkt_Cit1D AS NVARCHAR(50)),
    CAST(t.Tkt_Cit2D AS NVARCHAR(50)),
    t.Tkt_Datacreazione,
     CAST(t.Tkt_Codoperatore AS NVARCHAR(15)),
     'Meridiano'
FROM Meridiano.dbo.TKT AS t
INNER JOIN ReconciledDataLayer.dbo.Stations AS s
    ON t.Tkt_Cit1D = s.Name
WHERE NOT EXISTS (
    SELECT 1
    FROM Meridiano.dbo.VETTORI AS v
    WHERE v.Vettori_Cod = t.Tkt_Vettore
)
AND t.Tkt_Codhotel = '';
```



Here's the final schema for the reconciled data layer (please not that SSMS's drawing tool can be sometimes messy while representing all the tables with all the foreign keys so I told him to omit some of them in order to have a clearer view of the database structure).

In order to have a proper reconciliation between the data from different sources, the entity-relationship structure of the reconciled data layer have been slightly modified wrt the corporation's schemas (e.g. Meridiano did not feature a table to store train stations and had one big ticket table containing both all flights and train tickets, while Conti Travel's database featured different ticket tables for different carriers but it didn't have any table grouping the different packages/package categories, and so on).

Dimensional Fact Model

Following step consists in denormalizing the reconciled data in order to speed up the query performance by introducing a model where, besides the adding of redundant data, the number of joins between the tables is reduced. This can be done by converting the current schema into a *Star Schema*.

By looking at the reconciled data tables it is immediate to notice a deep hierarchy of tables (most clear example is the chain *Cities - Municipalities - Provinces - Regions - Countries - Areas*) that is proper of a *Snowflake Schema* whose purpose is to reduce the data redundancy, while in a *Star Schema* all of those information are collapsed into a large, unique table.

In order to convert the current model into a dimensional fact model we identify the facts (or the events) as the transactions (i.e. the sale of a ticket or reservation) and the dimensions (or the measures) as basically everything else (e.g. places, dates). The resulting model is more of a *"Constellation" Schema* than a *Star Schema* (it's a very bad catch, I know) since there are multiple tables who are suitable to become the central stars of such a schema.

Dimensional tables will have an ID working as key to connect to the Fact tables and will be:

- *Dimension Date*: in order to have a time reference useful to measure the facts in terms of occurrences and time intervals. A table registering all the days within a limited time interval is needed to have a scalable measure off all possible *day-month-year* combinations, without it in our dimensional data model we could represent only the days in which an event has occurred and not the ones without an associated fact, leading to a non-continuous time representation.

- *Dimension Geography*: collapses the long aforementioned chain into an unique table comprehensive of Cities, Municipalities, Provinces, Regions, Countries and Areas

- *Dimension Operator/Client*: is an ambivalent table used to store information regarding the operators that sell tickets/reservation and the clients who are often other smaller business travel agencies

- *Dimension Terminal*: collapses all the information about travel hubs like airports and train stations, having a field referenced to *Dimension Geography* in order to represent the city's coordinates

- *Dimension Airline*: is self-explaining at this point. Note: in this implementation the airline is a dimension but it could easily "degenerate" to an attribute of the *Fact Flight* table since the only information providing is the name of the airline and its IATA code

- *Dimension Hotel*: is also self-explaining

- *Dimension Accommodation*: unless one does not know what an hotel room accommodation is (single suite, double room, deluxe etc.) this is also self-explaining

Note: while importing the data from the operational sources, only the amounts calculated in Euro have been selected. This makes the adoptions of elements such as *Dimension Currency* pretty useless in this scenario.

Fact tables will differ from original tables because of the usage of several foreign keys to connect to the dimensional tables. Let those be:

- *Fact Reservation*: having a reference to the *Dimension Hotel*, *Dimension Date*, *Dimension Client* who requested the reservation, *Dimension Operator* who made the reservation and so on

- *Fact Ticket*: having reference to the date and terminal of departure/arrival, operator who made the ticket, amount etc.

- *Fact Flight/Segment*: consists in a subdivision of the measures of a flight, the first one being focused on financial aspects and the second one focused on the navigation ones. The "duality" of such tables derives from the fact that many flights have a series of intermediate stops and, in order to have a clean view on the reconciled layer, one table stores the information of a flight such as id, amount, tax, airline and the other table will have a set of rows for each flight storing each departure/arrival hub

Note: the dimensional fact model is missing the representation of *Fact Package* and *Dimension Category*, not because I'm not able to implement them but because they are tables with lots of attributes and measures, being time consuming converting them I choose to proceed by omitting them for two reasons:

- First one is the fact that in order to do a "scholastic" demonstration of how a Data Warehouse is implemented is largely sufficient to focus on tickets and reservations and the integration of packet mechanism would be only an enhancement in terms of complexity

- Second reason is the fact that I realized I'm not getting paid to do all of this stuff and, after wasting a discrete amount of time in the development of the project (plus having a lot of other things to do) I simply decide to keep it short

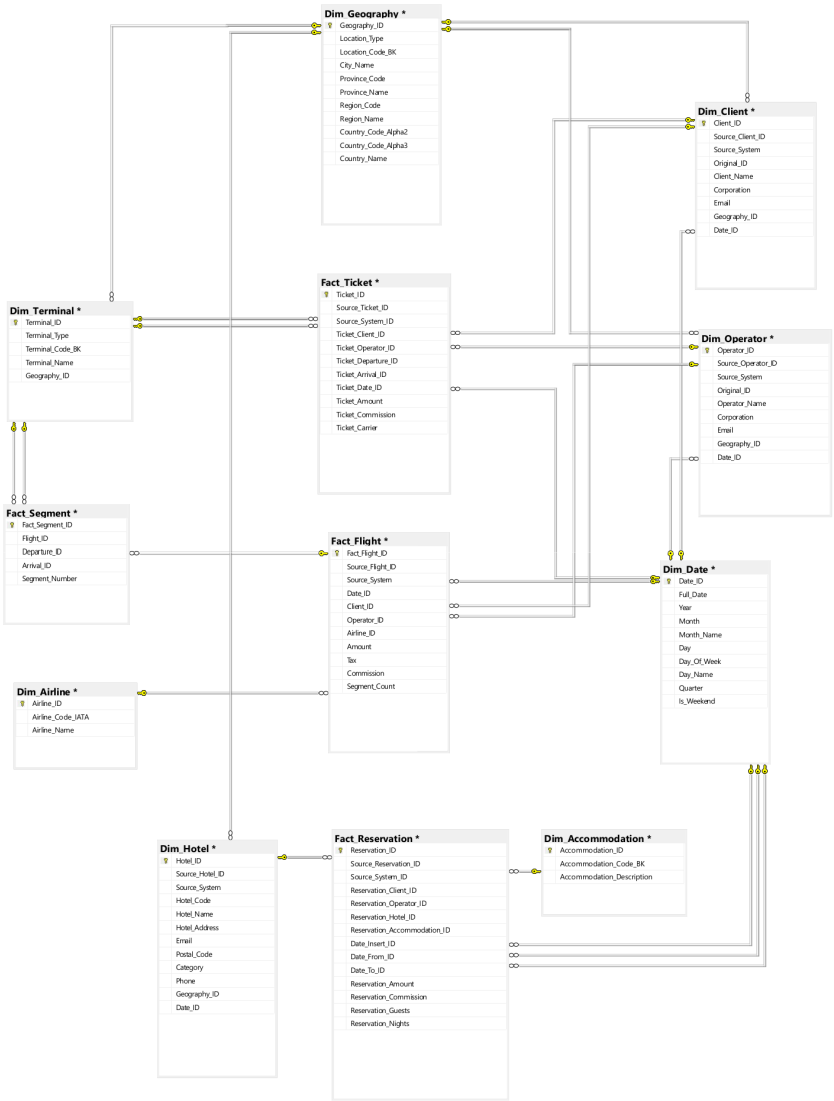
As for in the reconciled data layer section, here are a couple scripts used to build and populate the dimensional fact model along with a schema kindly provided by SSMS. Always, all of the ETL queries written and any material to be consulted are attached to this document.

```sql
USE DimensionalFactModel
GO
CREATE TABLE Dim_Hotel (
    Hotel_ID INT IDENTITY(1,1) PRIMARY KEY,
    -- Business Key composed by original id + source
    Source_Hotel_ID NVARCHAR(50),
    Source_System NVARCHAR(50),
    Hotel_Code NVARCHAR(50),
    Hotel_Name NVARCHAR(255),
    Hotel_Address NVARCHAR(255),
    Email NVARCHAR(255),
    Postal_Code NVARCHAR(50),
    Category NVARCHAR(50),
    Phone NVARCHAR(50),
    -- Link to Dim_Geography
    Geography_ID INT,
    -- Link to Dim_Date
    Date_ID INT
);
```

```sql
CREATE TABLE Dim_Geography (
    -- Data Warehouse Surrogate Key (Int, not Varchar!)
    Geography_ID INT IDENTITY(1,1) PRIMARY KEY,
    -- Data Origin (Municipality and Provinces are for Italy only!)
    Location_Type NVARCHAR(50),
    Location_Code_BK NVARCHAR(50),
    -- Area table has been bypassed (it mixes geography and events PLUS I'm not
        gonna need it in the DFM)
    City_Name NVARCHAR(255),
    Province_Code NVARCHAR(50),
    Province_Name NVARCHAR(255),
    Region_Code NVARCHAR(50),
    Region_Name NVARCHAR(255),
    Country_Code_Alpha2 NVARCHAR(50),
    Country_Code_Alpha3 NVARCHAR(50),
    Country_Name NVARCHAR(255)
);
```

```sql
-- Airports identified by IATA code
INSERT INTO Dim_Terminal (
    Terminal_Type,
    Terminal_Code_BK,
    Terminal_Name,
    Geography_ID
)
SELECT
    'Airport' AS Terminal_Type,
    TRIM(iata) AS Terminal_Code_BK,
    TRIM(name) AS Terminal_Name,
    g1.Geography_ID
FROM [ReconciledDataLayer].[dbo].[Airports] as a
LEFT JOIN Dim_Geography as g1
    ON TRIM(a.city) = g1.City_Name
    AND g1.Location_Type = 'City';
-- Stations identified by name (note: Italy only)
INSERT INTO Dim_Terminal (
    Terminal_Type,
    Terminal_Code_BK,
    Terminal_Name,
    Geography_ID
)
SELECT
    'Station' AS Terminal_Type,
    TRIM(name) AS Terminal_Code_BK,
    TRIM(name) AS Terminal_Name,
    g2.Geography_ID
FROM [ReconciledDataLayer].[dbo].[Stations] as s
LEFT JOIN Dim_Geography as g2
    ON TRIM(s.city) = g2.City_Name
    AND g2.Location_Type = 'City';
```

```sql
INSERT INTO Fact_Flight (
    Source_Flight_ID,
    Source_System,
    Date_ID,
    Client_ID,
    Operator_ID,
    Airline_ID,
    Amount,
    Tax,
    Commission,
    Segment_Count
)
SELECT
    TRIM(SUBSTRING(f.id, 3, LEN(f.id))) AS Source_Flight_ID,
    f.src AS Source_System,
    d.Date_ID,
    c.Client_ID,
    o.Operator_ID,
    a.Airline_ID,
    f.amount AS Amount,
    f.tax AS Tax,
    f.commission AS Commission,
    f.segments AS Segment_Count
FROM ReconciledDataLayer.dbo.Flights f
LEFT JOIN Dim_Client AS c
    ON TRIM(SUBSTRING(f.clientcode, 3, LEN(f.clientcode))) = c.Source_Client_ID AND
        f.src = c.Source_System
LEFT JOIN Dim_Operator AS o
    ON TRIM(SUBSTRING(f.operator, 3, LEN(f.operator))) = o.Source_Operator_ID AND f
        .src = o.Source_System
LEFT JOIN Dim_Airline AS a
    ON f.airline = a.Airline_Code_IATA
LEFT JOIN Dim_Date AS d
    ON CAST(f.date as DATE) = d.Full_Date;
```



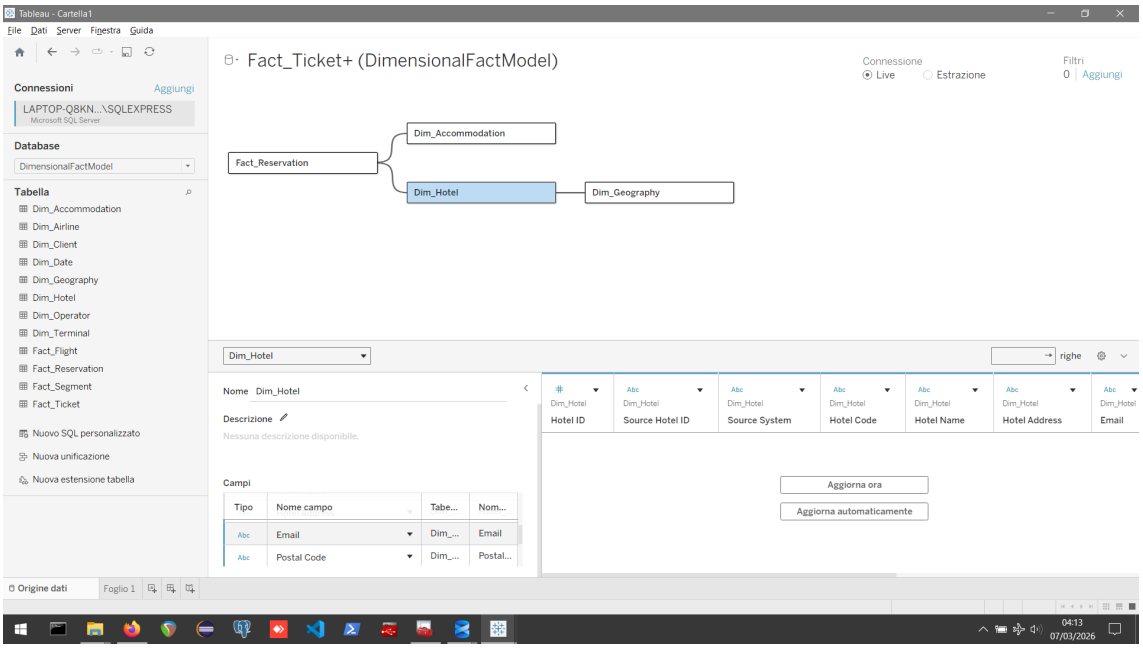
Here is the table schema for the dimensional fact model implementation as discussed before.

Data Marts

Being the schema a constellation one more than of a star one it is a good practice to virtually divide the warehousing layer into two or more *Data Marts*, each one having a certain scope and focus. In this case, three marts have been established:

- the first one built around the *Fact Reservation* along with dimensions as *Hotel*, *Accommodation*, *Date* and *Geography*
- the second one built around *Fact Ticket* along with dimensions like *Date*, *Operator*, *Terminal* and *Geography*
- the last one built around the *Facts Flight* and *Segment* having dimensions *Airline*, *Terminal* and so on

Each one of them is configured by simply instructing Tableau about how to link each foreign key between the fact tables and the dimension tables (for example, by adding the constraint $FactTicket.OperatorID = DimensionOperator.ID$ and doing the same with all the dimensions involved like *Date*, *Terminal* and *Geography*)



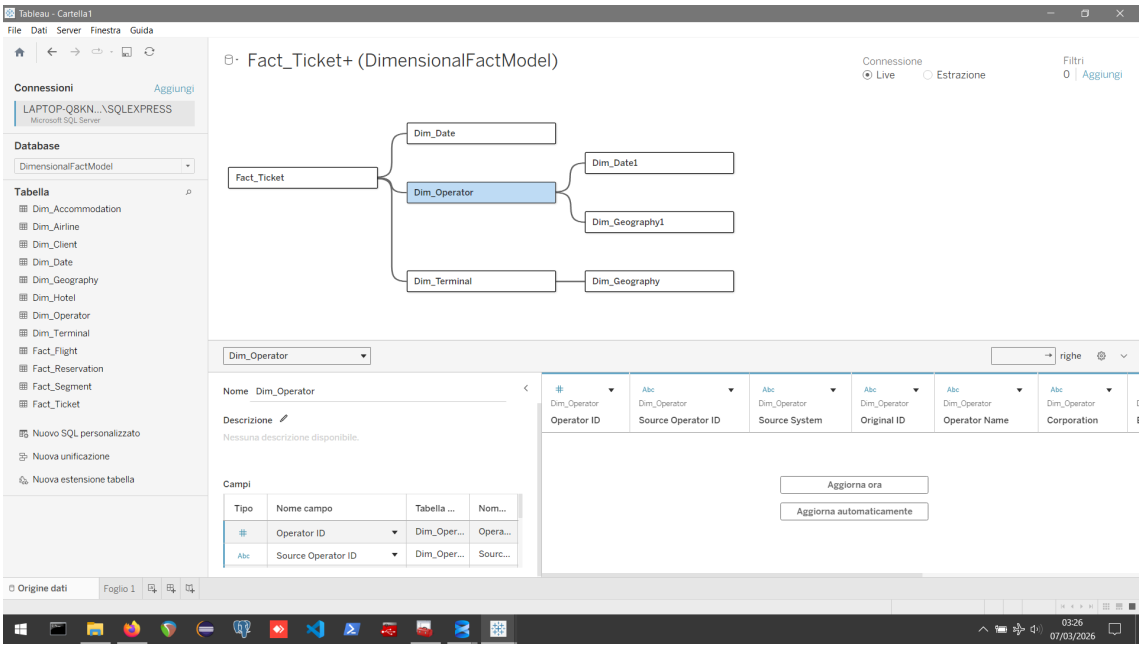


Tableau - Cartella1
File Dati Server Finestra Guida
Connessioni Aggiungi
LAPTOP-Q8KN...\SQLEXPRESS
Microsoft SQL Server
Database
DimensionalFactModel
Tabella
Dim_Accommodation
Dim_Airline
Dim_Client
Dim_Date
Dim_Geography
Dim_Hotel
Dim_Operator
Dim_Terminal
Fact_Flight
Fact_Reservation
Fact_Segment
Fact_Ticket
Nuovo SQL personalizzato
Nuova unificazione
Nuova estensione tabella
Fact_Ticket+ (DimensionalFactModel)
Connessione
Live
Estrazione
Filtri
0 Aggiungi
Fact_Ticket
Dim_Date
Dim_Operator
Dim_Date1
Dim_Geography1
Dim_Terminal
Dim_Geography
Dim_Operator
righe
Nome Dim_Operator
Descrizione
Nessuna descrizione disponibile.
Campi
Tipo
Nome campo
Tabella ...
Nom...
Operator ID
Dim_Oper...
Opera...
Source Operator ID
Dim_Oper...
Sourc...
Operator ID
Source Operator ID
Source System
Original ID
Operator Name
Corporation
Aggiorna ora
Aggiorna automaticamente
Origine dati
Foglio 1
03:26
07/03/2026

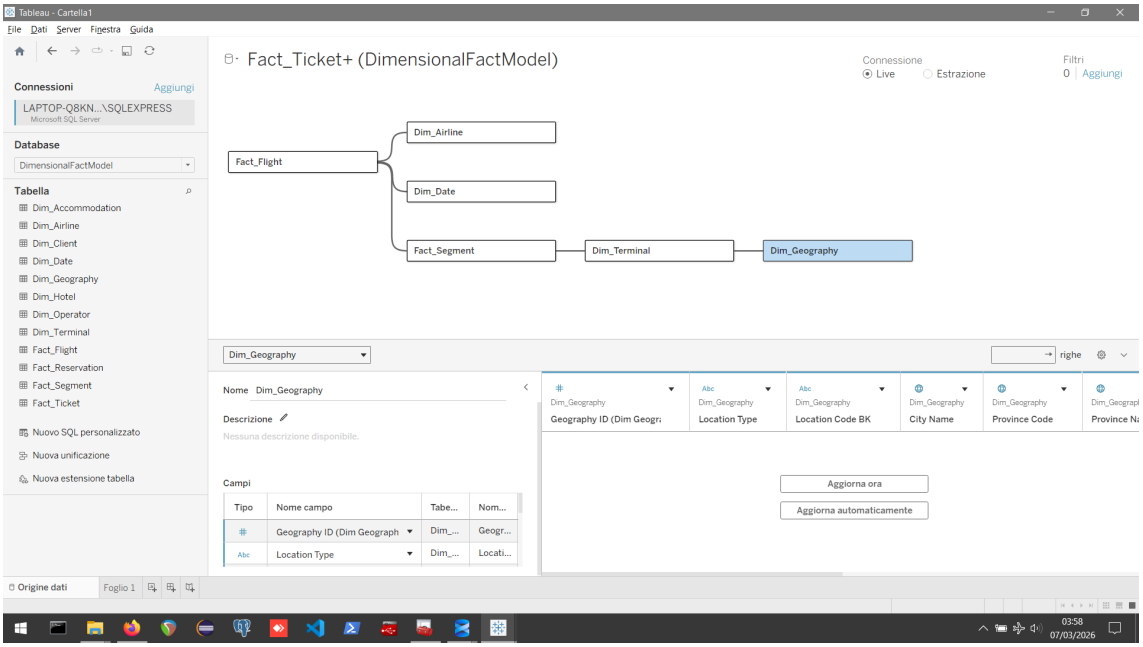


Tableau - Cartella1
File Dati Server Finestra Guida
Connessioni Aggiungi
LAPTOP-Q8KN...\SQLEXPRESS
Microsoft SQL Server
Database
DimensionalFactModel
Tabella
Dim_Accommodation
Dim_Airline
Dim_Client
Dim_Date
Dim_Geography
Dim_Hotel
Dim_Operator
Dim_Terminal
Fact_Flight
Fact_Reservation
Fact_Segment
Fact_Ticket
Nuovo SQL personalizzato
Nuova unificazione
Nuova estensione tabella
Fact_Ticket+ (DimensionalFactModel)
Connessione
Live
Estrazione
Filtri
0 Aggiungi
Fact_Flight
Dim_Airline
Dim_Date
Fact_Segment
Dim_Terminal
Dim_Geography
Dim_Geography
righe
Nome Dim_Geography
Descrizione
Nessuna descrizione disponibile.
Campi
Tipo
Nome campo
Tabe...
Nom...
Geography ID (Dim Geograph
Dim_...
Geogr...
Location Type
Dim_...
Locati...
Geography ID (Dim Geogr
Location Type
Location Code BK
City Name
Province Code
Aggiorna ora
Aggiorna automaticamente
Origine dati
Foglio 1
03:58
07/03/2026

Business Intelligence

Last but not least, once that Tableau has been told how to comfortably sit on the top of SSMS hosting the data marts, just drag and drop some metrics of interest and set some filters to analyze visually the outcome to certain queries that normally would be difficult to pronounce and involving a lot of (slow) joins between tables.

The following example queries are nothing exceptionally difficult but the process can always be made as complicated as you like.

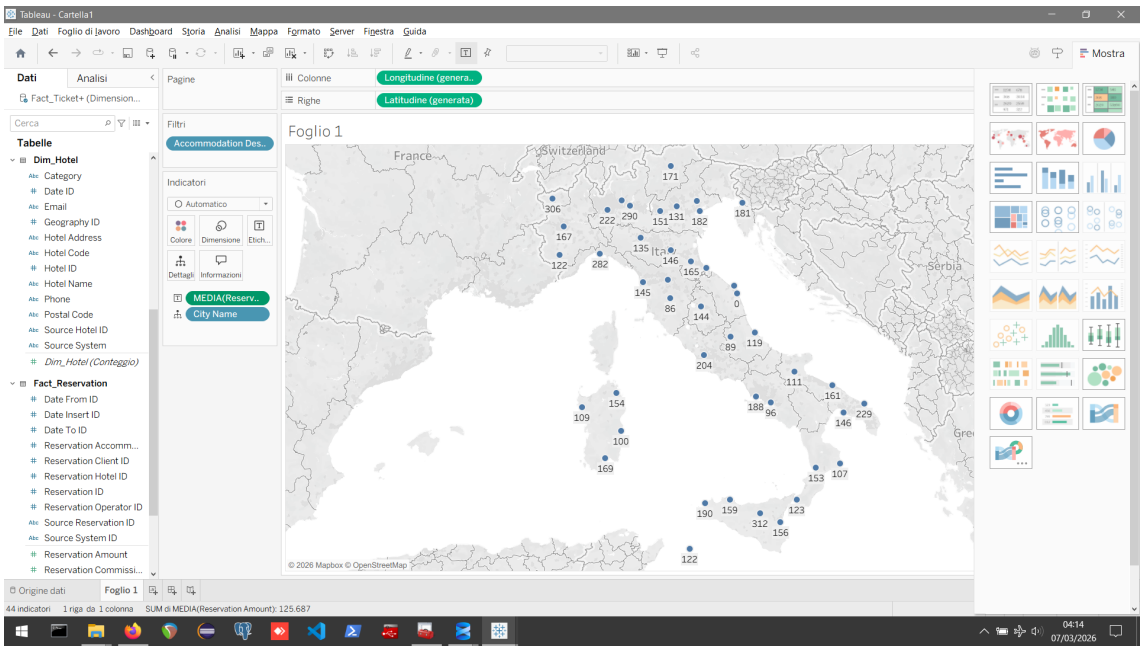


Figure 1: for every italian city, compute the average amount needed to reserve a double room (both for two guests or single use)

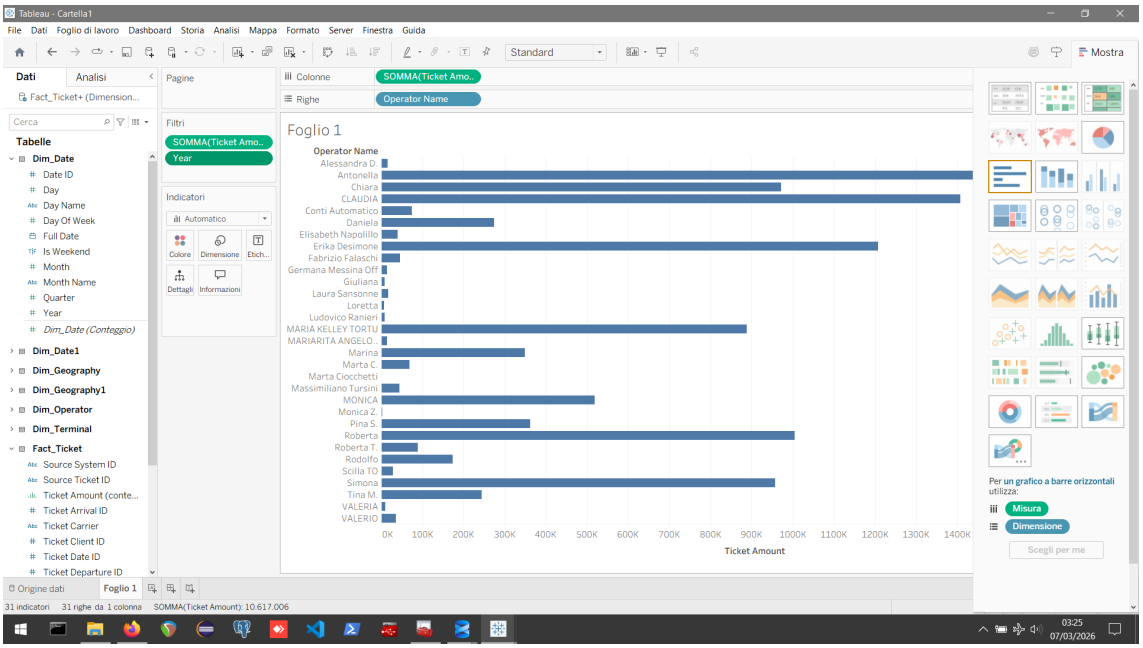


Figure 2: show the list of tour operators who sold over 2000 euros worth of tickets between years 2015 and 2020

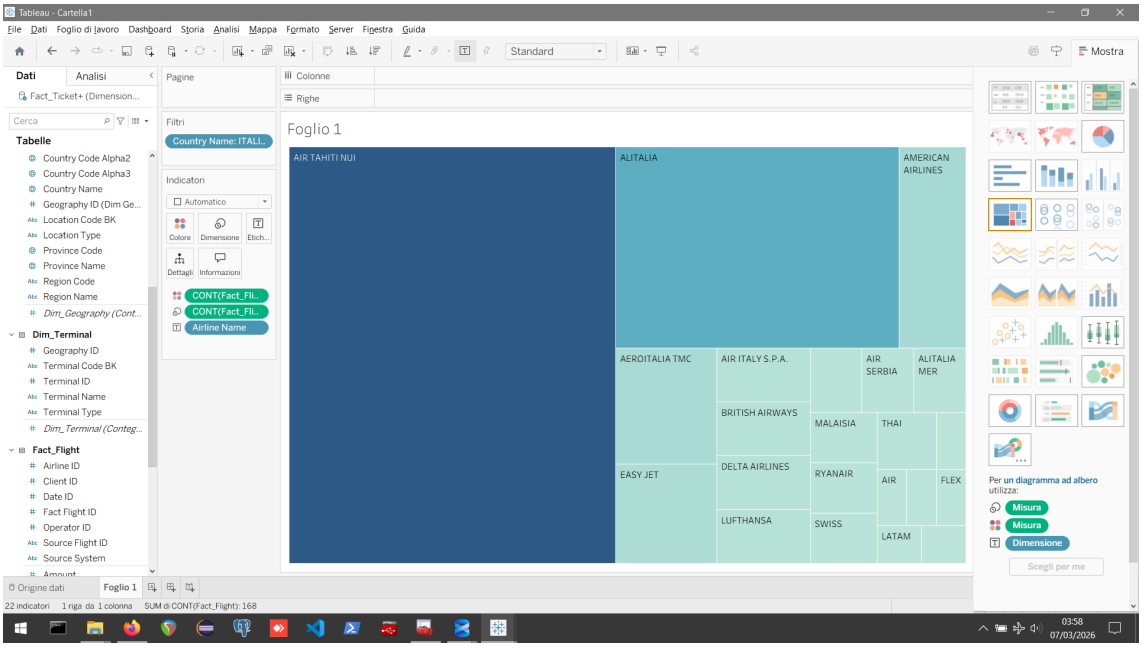


Figure 3: show the name and quantity of airlines whose flights have landed in Italy on a certain date